Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 28/07/2026 **Date of Submission: 28/07/2026**

Class: T1

Student Roll No: 2401036 **Student Name: Dhruv Hemdev**

Experiment 3: Orchestrating Multi-Container Applications Using Docker Compose

Aim:

To understand how Docker Compose is used to define, configure and manage a multi-container application from a single declarative file, and to perform the lifecycle operations of such a stack.

This experiment maps to LO4.

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory:

A single container is sufficient for a single process, but a realistic application is rarely a single process. A typical deployment couples a web server, an application runtime, a database and a cache, and each of these belongs in its own container so that it can be versioned, scaled and restarted independently. Managing such a set with plain docker run commands quickly becomes unwieldy, because every port mapping, environment variable, volume and network must be repeated by hand on every invocation and kept consistent between machines.

Docker Compose solves this by moving the entire specification into a single declarative YAML file, conventionally named docker-compose.yml. The file describes the desired end state of the application rather than the sequence of commands needed to reach it. Each entry under the services key defines one container: the image it is built from, the name it should be given, the ports published to the host, the environment variables injected into it, and the volumes mounted into its filesystem. A single command then reconciles the running system with that description.

Compose also supplies the surrounding infrastructure automatically. When a stack is brought up it creates a dedicated bridge network and attaches every service to it, so containers can address one another by service name instead of by IP address. Named volumes declared

under the top-level volumes key are created on demand and persist independently of the containers that use them, which is what allows database state to survive a container being removed and recreated. Because the YAML file is a plain text artifact, it can be committed to version control alongside the source code, which makes an entire environment reproducible by any developer who checks out the repository.

The commands used to drive a Compose project are summarised below.

Command	**Description**
docker compose up	Start all services
docker compose up -d	Start in detached mode
docker compose ps	Show running services
docker compose logs	View logs
docker compose stop	Stop services
docker compose start	Restart services
docker compose restart	Restart containers
docker compose down	Stop and remove containers
docker compose down -v	Remove containers and volumes
docker compose images	Show images
docker compose config	Validate Compose file

Implementation:

1. Create the Project Directory:

A Compose project is identified by the directory that contains its YAML file, and that directory name becomes the default project prefix applied to the network and to any named volumes. The command mkdir compose-demo creates a clean working directory so that the stack is isolated from any other Docker work on the machine.

```
dhruv@Dhruvs-MacBook-Air compose-demo % cd ~
dhruv@Dhruvs-MacBook-Air ~ % mkdir compose-demo
dhruv@Dhruvs-MacBook-Air ~ % cd compose-demo
dhruv@Dhruvs-MacBook-Air compose-demo %
```

2. Author the Compose File:

The specification is written to a file named docker-compose.yml inside that directory. Four services are declared. The web service publishes Nginx on host port 8080, the database service runs MySQL 8.0 with its root password, initial database and application user supplied as environment variables, the apache service publishes an Apache HTTP Server on host port 8081, and the cache service exposes Redis on its default port. A named volume, mysql-data, is declared at the bottom of the file and mounted at /var/lib/mysql so that the database contents outlive the container. MySQL is published on host port 3307 rather than 3306 because a MySQL server installed directly on the host was already bound to 3306; the container still listens on 3306 internally, and only the host-side half of the mapping changes.

```
version: '3.8'

services:

  web:
    image: nginx
    container_name: webserver
    ports:
      - "8080:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123
    ports:
      - "3307:3306"
    volumes:
      - mysql-data:/var/lib/mysql

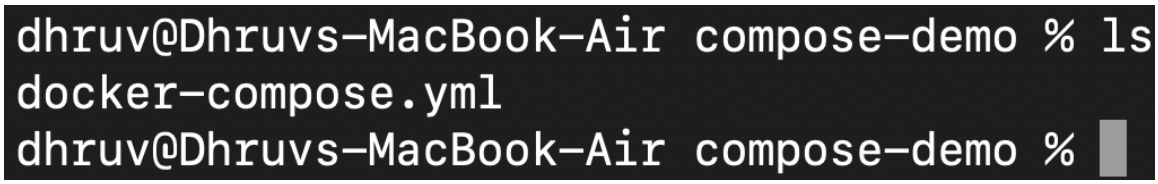
  apache:
    image: httpd
    container_name: apacheserver
    ports:
      - "8081:80"

  cache:
    image: redis
    container_name: redisserver
    ports:
      - "6379:6379"

volumes:
  mysql-data:
```

3. Confirm the File Was Saved:

Listing the directory verifies that the file was written with the exact name docker-compose.yml. The name matters, because Compose looks for that filename by default and reports that no configuration file was found if it has been saved with a different extension.



4. Review the Configuration:

Printing the file back with cat docker-compose.yml confirms that the indentation survived the save. YAML derives its structure entirely from indentation, so a single misaligned key silently changes which service an option belongs to, and reviewing the file before starting the stack is the cheapest way to catch such an error.

```
dhruv@Dhruvs-MacBook-Air compose-demo % cat docker-compose.yml
version: '3.8'

services:

  web:
    image: nginx
    container_name: webserver
    ports:
      - "8080:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123
    ports:
      - "3307:3306"
    volumes:
      - mysql-data:/var/lib/mysql

  apache:
    image: httpd
    container_name: apacheserver
    ports:
      - "8081:80"

  cache:
    image: redis
    container_name: redisserver
    ports:
      - "6379:6379"

volumes:
  mysql-data:
dhruv@Dhruvs-MacBook-Air compose-demo %
```

5. Validate the Compose File:

The command docker compose config parses the file and prints the fully resolved configuration that Compose will actually act on, with defaults filled in and every short-form value expanded. It reports that the version key is now obsolete, and it expands each port mapping into its explicit target, published, protocol and mode fields. Because it fails loudly on a malformed file, it is the standard way to validate a stack before starting it.

```
    container_name: redisserver
    image: redis
    networks:
      default: null
    ports:
      - mode: ingress
        target: 6379
        published: "6379"
        protocol: tcp
  database:
    container_name: mysqlserver
    environment:
      MYSQL_DATABASE: studentdb
      MYSQL_PASSWORD: student123
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_USER: student
    image: mysql:8.0
    networks:
      default: null
    ports:
      - mode: ingress
        target: 3306
        published: "3307"
        protocol: tcp
    volumes:
      - type: volume
        source: mysql-data
        target: /var/lib/mysql
        volume: {}
  web:
    container_name: webserver
    image: nginx
    networks:
      default: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
networks:
  default:
    name: compose-demo_default
volumes:
  mysql-data:
    name: compose-demo_mysql-data
dhruv@Dhruvs-MacBook-Air compose-demo %
```

6. Start All Services in Detached Mode:

The command docker compose up -d reconciles the running system with the file. Compose pulls any image that is not already cached, creates the compose-demo_default bridge network, creates the mysql-data volume, and then starts all four containers. The -d flag detaches them from the terminal so that they continue running in the background and the shell prompt is returned immediately.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose up -d
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
 remove it to avoid potential confusion
[+] up 5/5
 ✔ Network compose-demo_default Created
 ✔ Container webserver          Started
 ✔ Container redisserver        Started
 ✔ Container apacheserver       Started
 ✔ Container mysqlserver        Started

What's next:
    Filter, search, and stream logs from all your Compose services
    in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=compose-demo
dhruv@Dhruvs-MacBook-Air compose-demo %
```

7. List the Running Services:

Running docker compose ps reports the status of every container belonging to this project only, unlike docker ps which reports every container on the host. Each service is listed with its container name, image, current state and the host-to-container port mappings that were declared in the file.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose ps
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
 remove it to avoid potential confusion
NAME           IMAGE       COMMAND                  SERVICE    CREATED         STATUS         PORTS
apacheserver   httpd       "httpd-foreground"       apache     5 seconds ago   Up 4 seconds   0.0.0.0:8081->80/tcp,
:8081->80/tcp
mysqlserver    mysql:8.0   "docker-entrypoint.s…"   database   5 seconds ago   Up 4 seconds   0.0.0.0:3307->3306/tcp
:]:3307->3306/tcp
redisserver    redis       "docker-entrypoint.s…"   cache      5 seconds ago   Up 4 seconds   0.0.0.0:6379->6379/tcp
:]:6379->6379/tcp
webserver      nginx       "/docker-entrypoint.…"   web        5 seconds ago   Up 4 seconds   0.0.0.0:8080->80/tcp,
:8080->80/tcp
dhruv@Dhruvs-MacBook-Air compose-demo %
```

8. View the Aggregated Logs:

The command docker compose logs interleaves the standard output of all four services into a single stream, prefixing every line with the name of the service that produced it. The output is limited to the last few lines per service here so that it fits on one screen. Aggregating the logs makes it possible to correlate events across containers rather than inspecting each one separately.

```
mysqlserver  | 2026-07-28T05:54:26.816826Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Lo
ion '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
mysqlserver  | 2026-07-28T05:54:26.821330Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-addre
 '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock
mysqlserver  | 2026-07-28T05:54:26.821363Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. V
ion: '8.0.46'  socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GPL.
apacheserver  | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.20.0
 Set the 'ServerName' directive globally to suppress this message
apacheserver  | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.20.0
 Set the 'ServerName' directive globally to suppress this message
apacheserver  | [Tue Jul 28 05:54:26.463872 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.68 (Unix) conf
red -- resuming normal operations
apacheserver  | [Tue Jul 28 05:54:26.464062 2026] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGR
D'
redisserver   | 1:M 28 Jul 2026 05:54:26.494 * <ReJSON> Initialized shared string cache, thread safe: true.
redisserver   | 1:M 28 Jul 2026 05:54:26.494 * Module 'ReJSON' loaded from /usr/local/lib/redis/modules//rejson.so
redisserver   | 1:M 28 Jul 2026 05:54:26.494 * <search> Acquired RedisJSON_V7 API
redisserver   | 1:M 28 Jul 2026 05:54:26.494 * Server initialized
redisserver   | 1:M 28 Jul 2026 05:54:26.494 * Ready to accept connections tcp
redisserver   | 1:M 28 Jul 2026 05:54:26.494 # WARNING: Redis does not require authentication and is not protected by
twork restrictions. Redis will accept connections from any IP address on any network interface.
webserver     | 2026/07/28 05:54:26 [notice] 1#1: start worker process 33
webserver     | 2026/07/28 05:54:26 [notice] 1#1: start worker process 34
webserver     | 2026/07/28 05:54:26 [notice] 1#1: start worker process 35
webserver     | 2026/07/28 05:54:26 [notice] 1#1: start worker process 36
webserver     | 2026/07/28 05:54:26 [notice] 1#1: start worker process 37
webserver     | 2026/07/28 05:54:26 [notice] 1#1: start worker process 38

What's next:
    Filter, search, and stream logs from all your Compose services
    in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=compose-demo
dhruv@Dhruvs-MacBook-Air compose-demo %
```

9. Inspect the Database Service Individually:

Where the aggregated view is too noisy, docker logs mysqlserver narrows the output to a single container. The closing lines of the startup sequence show InnoDB completing its initialisation, the server configuring TLS for its main channel, and finally mysqld announcing version 8.0.46 ready for connections on port 3306, which is the point at which the database is usable.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker logs mysqlserver --tail 10
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
2026-07-28T05:54:26.740174Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be
moved in a future release. Please use SET GLOBAL host_cache_size=0 instead.
2026-07-28T05:54:26.740527Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.46) starting as process 1
2026-07-28T05:54:26.742484Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-07-28T05:54:26.775383Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-07-28T05:54:26.816075Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-07-28T05:54:26.816093Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted c
ections are now supported for this channel.
2026-07-28T05:54:26.816826Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/ru
ysqld' in the path is accessible to all OS users. Consider choosing a different directory.
2026-07-28T05:54:26.821330Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port:
60, socket: /var/run/mysqld/mysqlx.sock
2026-07-28T05:54:26.821363Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.46'
socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GPL.

What's next:
    View and search logs for all containers in one place
    with Docker Desktop's Logs view. docker-desktop://dashboard/logs
dhruv@Dhruvs-MacBook-Air compose-demo %
```

10. Show the Images Backing the Stack:

The command docker compose images lists the image and tag resolved for each service together with its identifier and size. It confirms which concrete image the stack is actually running, which matters for services such as web and cache that were declared without an explicit tag and therefore resolve to latest.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose images
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, ple
 remove it to avoid potential confusion
CONTAINER           REPOSITORY          TAG                 PLATFORM            IMAGE ID            SIZE
CREATED
apacheserver        httpd               latest              linux/arm64/v8      305fd8326a27        45.5MB
2 weeks ago
mysqlserver         mysql               8.0                 linux/arm64/v8      7dcddc01f13b        229MB
2 months ago
redisserver         redis               latest              linux/arm64/v8      c88d347edef6        54.1MB
3 days ago
webserver           nginx               latest              linux/arm64/v8      5a88c9c45479        61.4MB
12 days ago
dhruv@Dhruvs-MacBook-Air compose-demo %
```

11. Connect to the Database:

Executing docker exec -it mysqlserver mysql -u student -p opens an interactive MySQL client inside the running container using the credentials defined in the Compose file. The connection is made over the container's own socket rather than the published host port. SHOW DATABASES confirms that studentdb was created during initialisation, and USE studentdb selects it for the rest of the session.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker exec -it mysqlserver mysql -u student -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| performance_schema |
| studentdb          |
+--------------------+
3 rows in set (0.00 sec)

mysql> USE studentdb;
Database changed
mysql>
```

12. Create a Table and Insert a Row:

Inside the selected database, CREATE TABLE defines a students table with a roll number as its primary key, and INSERT adds a single row. Both statements report Query OK, showing that the student user was granted full privileges on studentdb by the entrypoint script, which is the behaviour MYSQL_DATABASE and MYSQL_USER together are meant to produce.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker exec -it mysqlserver mysql -u student -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE studentdb;
Database changed
mysql> CREATE TABLE students (roll INT PRIMARY KEY, name VARCHAR(50), batch VARCHAR(20));
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO students VALUES (2401036, 'Dhruv Hemdev', 'T1');
Query OK, 1 row affected (0.01 sec)

mysql>
```

13. Query the Table:

Reconnecting to the same container and running SHOW TABLES lists the students table, and SELECT * returns the row inserted earlier. Because the data directory is backed by the mysql-data volume rather than by the container's writable layer, this row would survive the container being removed and recreated.

```
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE studentdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+--------------------+
| Tables_in_studentdb |
+--------------------+
| students           |
+--------------------+
1 row in set (0.01 sec)

mysql> SELECT * FROM students;
+---------+-------------+-------+
| roll    | name        | batch |
+---------+-------------+-------+
| 2401036 | Dhruv Hemdev | T1    |
+---------+-------------+-------+
1 row in set (0.00 sec)

mysql>
```

14. Test the Nginx Web Service:

Requesting http://localhost:8080 with curl exercises the port mapping declared for the web service. The default Nginx welcome page is returned, which confirms that the container is serving traffic and that host port 8080 is correctly forwarded to port 80 inside the container.

```
dhruv@Dhruvs-MacBook-Air compose-demo % curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
dhruv@Dhruvs-MacBook-Air compose-demo %
```

15. Test the Apache Web Service:

Requesting http://localhost:8081 returns the Apache HTTP Server default page. Both web services listen on port 80 internally, yet they coexist without conflict because each is published on a different host port. This illustrates how network namespacing lets identical services run side by side on one machine.

```
dhruv@Dhruvs-MacBook-Air compose-demo % curl http://localhost:8081
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
dhruv@Dhruvs-MacBook-Air compose-demo %
```

16. Test the Redis Service:

The command docker exec -it redisserver redis-cli ping sends a health-check command to the cache service from within its own container. The reply PONG indicates that the Redis server has started and is accepting commands.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker exec -it redisserver redis-cli ping
PONG

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug redisserver
    Learn more at https://docs.docker.com/go/debug-cli/
dhruv@Dhruvs-MacBook-Air compose-demo %
```

17. Stop the Services:

The command docker compose stop halts every container in the project but leaves the containers, the network and the volumes in place. This is the correct way to pause a stack temporarily, because nothing is destroyed and the containers keep their identity and their writable layers.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose stop
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
 remove it to avoid potential confusion
[+] stop 4/4
 ✔ Container apacheserver Stopped
 ✔ Container webserver    Stopped
 ✔ Container mysqlserver  Stopped
 ✔ Container redisserver  Stopped
dhruv@Dhruvs-MacBook-Air compose-demo %
```

18. Start the Services Again:

Because the containers still exist, docker compose start restarts the same containers rather than creating new ones. No image is pulled and no initialisation is repeated, so the database comes back with the students table still present.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose start
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
 remove it to avoid potential confusion
[+] start 4/4
 ✔ Container redisserver  Started
 ✔ Container mysqlserver  Started
 ✔ Container webserver    Started
 ✔ Container apacheserver Started
dhruv@Dhruvs-MacBook-Air compose-demo %
```

19. Restart the Containers:

The command docker compose restart performs a stop followed by a start in a single operation. It is the usual way to make a container pick up a configuration change that is read at process start-up without recreating the container itself.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose restart
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
 remove it to avoid potential confusion
[+] restart 0/4
 ⠿ Container apacheserver Restarting
 ⠿ Container redisserver  Restarting
 ⠿ Container webserver    Restarting
 ⠿ Container mysqlserver  Restarting
dhruv@Dhruvs-MacBook-Air compose-demo %
```

20. Stop and Remove the Containers:

The command docker compose down stops all four containers, removes them, and deletes the network that Compose created. Named volumes are deliberately left behind, so the data written to mysql-data survives this operation.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose down
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, ple
 remove it to avoid potential confusion
[+] down 5/5
 ✔ Container mysqlserver        Removed                                                                          0
 ✔ Container apacheserver       Removed                                                                          1
 ✔ Container redisserver        Removed                                                                          0
 ✔ Container webserver          Removed                                                                          0
 ✔ Network compose-demo_default Removed                                                                          0
dhruv@Dhruvs-MacBook-Air compose-demo %
```

21. Remove the Containers and the Volume:

Adding the -v flag, docker compose down -v additionally removes the named volumes belonging to the project. The output confirms that compose-demo_mysql-data was removed, which discards the database contents permanently and returns the machine to the state it was in before the experiment began.

```
dhruv@Dhruvs-MacBook-Air compose-demo % docker compose down -v
WARN[0000] /Users/dhruv/compose-demo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, ple
 remove it to avoid potential confusion
[+] down 1/1
 ✔ Volume compose-demo_mysql-data Removed                                                                        0
dhruv@Dhruvs-MacBook-Air compose-demo %
```

Conclusion:

In this experiment, a four-service application consisting of Nginx, MySQL, Apache and Redis was defined in a single docker-compose.yml file and managed as one unit. The declarative approach replaced what would otherwise have been four separate docker run invocations, each carrying its own port mappings, environment variables and volume arguments. The file was validated with docker compose config, the whole stack was created with docker compose up -d, and it was then inspected using docker compose ps, docker compose logs and docker compose images, which showed how Compose scopes its commands to the containers belonging to the project rather than to every container on the host.

Each service was then verified independently. The two web servers responded on ports 8080 and 8081, confirming that identical internal ports can be published side by side without conflict, and the Redis ping returned PONG. Inside the database container a table was created, a row inserted and then read back, which proved that the credentials declared in the file had been applied during initialisation. The lifecycle commands were then exercised in turn: stop and start reused the existing containers, restart combined the two, down removed the containers and the network while preserving the named volume, and down -v removed the volume as well. Taken together, the experiment established how Docker Compose turns an entire multi-container environment into a single version-controllable artifact whose lifecycle can be driven with one command at a time.